

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	V Yogeswar	Reg. No.:	192425175
Section / Batch:	AI & ML / 2024	Date:	

Code of Conduct

I V Yogeswar (192425175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V Yogeswar

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	15	15	18	12	15	75	75

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURUBARAN. T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

To recreate the Hidden Markov Model (HMM) without using any built-in NLP or HMM libraries.

Code:-

```
# Hidden Markov Model (HMM) from Scratch
# No built-in NLP or HMM libraries used

# -----
# Training Corpus
# -----

training_data = [
    ("The", "boy", "eats", "rice", ["DT", "NN", "VBZ", "NN"]),
    ("The", "girl", "drinks", "milk", ["DT", "NN", "VBZ", "NN"]),
    ("A", "cat", "drinks", "milk", ["DT", "NN", "VBZ", "NN"]),
    ("The", "dog", "chases", "cat", ["DT", "NN", "VBZ", "NN"]),
    ("A", "teacher", "teaches", "students", ["DT", "NN", "VBZ", "NNS"]),
    ("Students", "study", "English", ["NNS", "VBP", "NN"]),
    ("Birds", "fly", "high", ["NNS", "VBP", "RB"]),
    ("Children", "play", "games", ["NNS", "VBP", "NNS"])
]

# -----
# Task 1: Separate words and POS tags
# -----

print("\nTraining Sentences\n")

for words, tags in training_data:
    print("Words :", words)
    print("Tags :", tags)
    print()
```

```

# -----
# Task 2: Emission Probability
# -----

emission_count = {}
tag_count = {}

for words, tags in training_data:
    for word, tag in zip(words, tags):

        if tag not in emission_count:
            emission_count[tag] = {}

        emission_count[tag][word] = emission_count[tag].get(word, 0) + 1
        tag_count[tag] = tag_count.get(tag, 0) + 1

emission_prob = {}

for tag in emission_count:
    emission_prob[tag] = {}

    for word in emission_count[tag]:
        emission_prob[tag][word] = emission_count[tag][word] / tag_count[tag]

print("\nEmission Probability\n")

for tag in emission_prob:
    for word in emission_prob[tag]:
        print(f"P({word} | {tag}) = {emission_prob[tag][word]:.3f}")

# -----
# Task 3: Transition Probability
# -----

```

```
transition_count = {}
```

```
transition_total = {}
```

```
for words, tags in training_data:
```

```
    previous = "START"
```

```
    for tag in tags:
```

```
        if previous not in transition_count:
```

```
            transition_count[previous] = {}
```

```
            transition_count[previous][tag] = transition_count[previous].get(tag, 0) + 1
```

```
            transition_total[previous] = transition_total.get(previous, 0) + 1
```

```
        previous = tag
```

```
transition_prob = {}
```

```
for previous in transition_count:
```

```
    transition_prob[previous] = {}
```

```
    for current in transition_count[previous]:
```

```
        transition_prob[previous][current] = (
            transition_count[previous][current] /
            transition_total[previous]
        )
```

```
print("\nTransition Probability\n")
```

```
for previous in transition_prob:
```

```
    for current in transition_prob[previous]:
```

```

        print(f"P({current}|{previous}) = {transition_prob[previous][current]:.3f}")

# -----
# Task 4: Hidden Markov Model
# -----

print("\nHidden Markov Model Created Successfully.")

# -----
# Task 5: Viterbi Algorithm
# -----

test_sentence = ["The", "cat", "drinks", "milk"]

states = list(tag_count.keys())

viterbi = []
path = {}

for i, word in enumerate(test_sentence):

    viterbi.append({})

    new_path = {}

    for state in states:

        emission = emission_prob.get(state, {}).get(word, 0.0001)

        if i == 0:

            transition = transition_prob.get("START", {}).get(state, 0.0001)

            viterbi[i][state] = transition * emission

```

```

        new_path[state] = [state]

    else:

        best_prob = -1
        best_state = None

        for previous in states:

            transition = transition_prob.get(previous, {}).get(state, 0.0001)

            probability = viterbi[i-1][previous] * transition * emission

            if probability > best_prob:
                best_prob = probability
                best_state = previous

        viterbi[i][state] = best_prob
        new_path[state] = path[best_state] + [state]
    path = new_path

# -----
# Task 6: Predict POS Tags
# -----

best_state = max(viterbi[-1], key=viterbi[-1].get)
predicted_tags = path[best_state]

print("\nNew Input Sentence")
print(" ".join(test_sentence))

print("\nPredicted POS Tags")
print(predicted_tags)

print("\nFinal Output\n")
print("{:<10} {:<10}".format("Word", "POS Tag"))
for word, tag in zip(test_sentence, predicted_tags):
    print("{:<10} {:<10}".format(word, tag))

```


Output:-

Training Sentences

Words : ['The', 'boy', 'eats', 'rice']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Words : ['The', 'girl', 'drinks', 'milk']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Words : ['A', 'cat', 'drinks', 'milk']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Words : ['The', 'dog', 'chases', 'cat']

Tags : ['DT', 'NN', 'VBZ', 'NN']

Words : ['A', 'teacher', 'teaches', 'students']

Tags : ['DT', 'NN', 'VBZ', 'NNS']

Words : ['Students', 'study', 'English']

Tags : ['NNS', 'VBP', 'NN']

Words : ['Birds', 'fly', 'high']

Tags : ['NNS', 'VBP', 'RB']

Words : ['Children', 'play', 'games']

Tags : ['NNS', 'VBP', 'NNS']

Emission Probability

$P(\text{The}|\text{DT}) = 0.600$

$P(\text{A}|\text{DT}) = 0.400$

$P(\text{boy}|\text{NN}) = 0.100$

$P(\text{rice}|\text{NN}) = 0.100$

$P(\text{girl}|\text{NN}) = 0.100$
 $P(\text{milk}|\text{NN}) = 0.200$
 $P(\text{cat}|\text{NN}) = 0.200$
 $P(\text{dog}|\text{NN}) = 0.100$
 $P(\text{teacher}|\text{NN}) = 0.100$
 $P(\text{English}|\text{NN}) = 0.100$
 $P(\text{eats}|\text{VBZ}) = 0.200$
 $P(\text{drinks}|\text{VBZ}) = 0.400$
 $P(\text{chases}|\text{VBZ}) = 0.200$
 $P(\text{teaches}|\text{VBZ}) = 0.200$
 $P(\text{students}|\text{NNS}) = 0.200$
 $P(\text{Students}|\text{NNS}) = 0.200$
 $P(\text{Birds}|\text{NNS}) = 0.200$
 $P(\text{Children}|\text{NNS}) = 0.200$
 $P(\text{games}|\text{NNS}) = 0.200$
 $P(\text{study}|\text{VBP}) = 0.333$
 $P(\text{fly}|\text{VBP}) = 0.333$
 $P(\text{play}|\text{VBP}) = 0.333$
 $P(\text{high}|\text{RB}) = 1.000$

Transition Probability

$P(\text{DT}|\text{START}) = 0.625$
 $P(\text{NNS}|\text{START}) = 0.375$
 $P(\text{NN}|\text{DT}) = 1.000$
 $P(\text{VBZ}|\text{NN}) = 1.000$
 $P(\text{NN}|\text{VBZ}) = 0.800$
 $P(\text{NNS}|\text{VBZ}) = 0.200$
 $P(\text{VBP}|\text{NNS}) = 1.000$
 $P(\text{NN}|\text{VBP}) = 0.333$
 $P(\text{RB}|\text{VBP}) = 0.333$
 $P(\text{NNS}|\text{VBP}) = 0.333$

Hidden Markov Model Created Successfully.

New Input Sentence

The cat drinks milk

Predicted POS Tags

['DT', 'NN', 'VBZ', 'NN']

Final Output

Word	POS Tag
------	---------

The	DT
-----	----

cat	NN
-----	----

drinks	VBZ
--------	-----

milk	NN
------	----